Cloudy Fleet Management UI/UX Design Blueprint

1. Executive Summary

Cloudy should adopt a **hybrid Operations Console and Fleet Command Center** design:

- The everyday workspace should be a dense, structured operations console optimized for dispatching, filtering, scheduling, and resolving conflicts.
- The dashboard should function as a lightweight command center focused on exceptions, capacity, delays, and operational risk.
- Manager-facing screens should emphasize trends, workload, fleet utilization, auditability, and unresolved issues.
- Dispatcher-facing screens should emphasize current assignments, availability, imminent departures, unassigned trips, and fast editing.

The primary UX objective is not to make every record visually prominent. It is to make the **next operational decision obvious**.

The recommended interface therefore centers on:

1. An exception-driven dashboard.
2. A unified trip operations workspace with list, schedule, and availability views.
3. A structured trip creation flow with persistent context and pre-submit validation.
4. Driver and vehicle availability embedded directly into assignment workflows.
5. Dense but configurable tables for operational records.
6. Side panels for quick inspection and full pages for complex editing.
7. Explicit role- and permission-aware actions.
8. A semantic status system that never relies on color alone.
9. Saved views, keyboard support, and persistent filters for frequent users.
10. A reusable token-based design system supporting light, dark, and high-contrast modes.

The current repository already contains the main conceptual building blocks—dashboard drill-in, trip list/calendar/map views, trip detail tabs, vehicle status history, employee availability, and role-filtered navigation—but these are consolidated into large components and backed by temporary in-memory state. The UX design should preserve the useful concepts while reorganizing them into clearer page-level workflows.

---

2. Assumptions About the Application

Confirmed requirements

The following should be treated as fixed:

- Cloudy’s operational MVP is the **Trip Scheduling workspace**.

- Inventory and Billing remain visible but disabled as “Coming Soon.”
- Official platform roles are SuperAdmin, Admin, Dispatcher, Encoder, and Viewer.
- Supabase Auth owns authentication credentials.
- Trips, trucks, employees, and users use cancellation, deactivation, or archival rather than hard deletion.
- Lookup tables are authoritative for statuses, roles, load types, and modules.
- Overlapping active driver or truck assignments must be blocked.
- Assignment changes, status changes, and relevant administrative actions must create history or audit records.
- Search and filtering are mandatory for trips, trucks, and employees.
- Trip events and fuel logs have backend support in the MVP, but do not require full standalone management modules.
- Real-time GPS, a driver mobile app, automated billing, and multi-tenant operations are outside the MVP.

Design assumptions

Assumption	Why it matters	Safest initial decision
“Manager” is a persona, not a separate platform role	The official platform roles do not include Manager, although Manager exists as an employee role	Map the manager persona primarily to Admin and SuperAdmin dashboard experiences
Most dispatcher work occurs on desktop	Scheduling requires simultaneous visibility of trips, availability, filters, and details	Optimize for 1280–1920px widths, then provide a usable tablet adaptation
A trip can initially be unassigned	The PRD permits nullable driver and truck assignments if the business process allows it	Support “Save as unassigned” while making the exception highly visible
“Delayed” and “overdue” are derived conditions	They are not authoritative trip statuses in the PRD	Display them as exception indicators, not new database statuses
Customer management is reference-data administration	Clients, internal codes, consignees, and locations exist, but no major customer operations module is defined	Place these under an Admin-only Reference Data area
Reports are limited in the MVP	Exact PDF and CSV formats remain unresolved	Provide dashboard drilldowns and on-screen analysis; defer a report-builder interface
Notifications initially reflect system data	Email, SMS, and push rules are not approved	Implement an in-app operational alert center without implying external delivery

Open business questions that affect design

1. What time window defines an overlapping assignment?
2. Which status transitions are permitted from each trip state?
3. What makes a Scheduled trip “stale” or delayed?
4. Can truck-capacity violations be overridden, or are they blocked?
5. Should cancelled trips appear in default operational views?
6. Are inactive drivers and vehicles visible as historical values when inspecting older trips?
7. Should Trip Management and Trip Schedule remain separate navigation destinations?
8. Is the helper limit permanently two, or only an MVP limitation?

These questions should be resolved before high-fidelity validation rules and status-transition controls are finalized.

---

3. Recommended Information Architecture

Application shell

Use a three-part desktop shell:

1. **Collapsible left navigation**
2. **Persistent top utility bar**
3. **Main workspace with an optional contextual right panel**

The current application uses a Hub, AppNavbar, Sidebar, and string-based view switching. That structure can evolve into a clearer hierarchy without changing the MVP feature set.

Hub

The Hub should remain a module-selection page after authentication.

- **Trip Scheduling**: enabled, prominent, labeled “Open workspace.”
- **Inventory**: disabled card, labeled “Coming Soon.”
- **Billing**: disabled card, labeled “Coming Soon.”
- Disabled modules should not appear selectable through keyboard focus as functional links.
- Include a concise description of each module’s purpose.
- Remember the last active workspace after login when technically appropriate.

Primary navigation

Operations

- Dashboard
- Trips
- Operations
- Schedule
- Create trip

- Driver availability
- Vehicle availability
- Fleet
- Vehicles
- Vehicle status history
- People
- Employees

Management

- Customers and reference data
- Analytics
- Operational alerts

Administration

- Users and roles
- Audit log
- System settings

Footer utility area

- Return to Hub
- Theme and contrast
- Help
- Current user and role
- Sign out

Navigation behavior

- Hide pages the user cannot read.
- Hide actions that have no relevance to the user's role.
- Show disabled actions only when their presence teaches the user something important, such as "Deactivation requires Admin access."
- Use tooltips or short supporting text for disabled administrative actions.
- Do not show dozens of locked navigation items to low-permission users.
- Preserve the selected section and page during navigation.
- Use page URLs and deep-linkable record states in the production architecture so browser back, refresh, and bookmarked records behave predictably.

Recommended hierarchy

```
Cloudy
├── Hub
└── Trip Scheduling
    ├── Dashboard
    ├── Trips
    │   ├── Operations
```

```
    │   ├── Schedule
    │   ├── Create trip
    │   ├── Driver availability
    │   └── Vehicle availability
    ├── Fleet
    │   ├── Vehicles
    │   └── Status history
    ├── People
    │   └── Employees
    ├── Management
    │   ├── Customers & reference data
    │   ├── Analytics
    │   └── Operational alerts
    └── Administration
        ├── Users & roles
        ├── Audit log
        └── System settings
```

---

4. Manager User Journey

The manager persona should primarily review operational health, investigate exceptions, and intervene where capacity or performance is at risk.

Start of day

1. Open the Dashboard.
2. Review:
3. Trips due today
4. Unassigned trips
5. Trips at risk
6. Trucks in maintenance
7. Available vehicle capacity
8. Unresolved operational alerts
9. Open an exception queue rather than scanning every trip.
10. Drill into affected trips through a details panel.
11. Assign follow-up work or change priority through existing operational controls.

During operations

1. Monitor Scheduled and In Progress counts.
2. Review delayed or stale derived conditions.
3. Inspect reassignment and vehicle-status histories.

4. Compare workload by day, client, truck state, or dispatcher.
5. Review changes made since the previous session.

End of day

1. Review incomplete or unresolved trips.
2. Check trips missing completion information.
3. Inspect cancellations, rescues, transfers, and backloads.
4. Review fleet availability for the following day.
5. Export or capture permitted operational summaries when export functionality is available.

Manager dashboard emphasis

- Operational health scorecard
- Exceptions requiring intervention
- Fleet utilization and maintenance
- Trip completion and cancellation trends
- Fuel-cost overview
- Recent assignment and status changes
- Audit and compliance visibility

The manager should not be forced through the same high-frequency scheduling interface used by dispatchers unless actively intervening.

---

5. Dispatcher User Journey

The dispatcher persona should have a fast, queue-oriented workflow.

Start of shift

1. Open the Trips workspace rather than a purely analytical dashboard.
2. Load a saved view such as:
3. Today’s departures
4. Unassigned
5. In progress
6. Requires attention
7. Review driver and truck availability.
8. Assign assets through an inline action or contextual panel.
9. Resolve conflicts before trips become operationally urgent.

Creating a trip

1. Select **Create trip**.
2. Enter trip identity, client, date, and load.

3. Add ordered pickup and drop-off stops.
4. Select or defer asset assignment.
5. Review availability and conflicts without leaving the form.
6. Review a summary.
7. Save as a draft or create the trip, depending on permission and business state.

Updating a trip

1. Search by trip code, client, driver, plate number, or consignee.
2. Open quick details in a side panel.
3. Change status, assignment, or selected operational fields.
4. See the effect before confirming:
5. Released assignment
6. New vehicle state
7. Conflict
8. Audit entry
9. Receive a clear success state and retain list context.

Dispatcher workspace emphasis

- Today and next 24 hours
- Unassigned trips
- Active conflicts
- Available drivers and trucks
- Rapid status changes
- Recent activity
- Keyboard-friendly search and actions
- Minimal navigation away from the current queue

---

6. Recommended Pages and Screens

Page	Main goal	Primary action	Core components
Login	Authenticate securely	Sign in	Email/password fields, password recovery link, session error
Hub	Enter an available module	Open Trip Scheduling	Module cards, disabled future modules
Dashboard	Understand operational health	Open an exception queue	KPI strip, exception list, schedule preview, fleet summary

Page	Main goal	Primary action	Core components
Trip Operations	Manage the dispatch queue	Create or assign trip	Dense table, saved views, filters, detail panel
Trip Schedule	Understand time and capacity	Create trip	Calendar/timeline, resource indicators, filters
Create/Edit Trip	Build a valid trip advice record	Save trip	Sectioned form, sticky summary, availability panel
Trip Details	Understand and manage one trip	Context-dependent status or assignment action	Header, route, assignments, events, fuel, activity
Driver Availability	Find assignable drivers	Assign driver	Availability table/timeline, conflict details
Vehicle Availability	Find assignable vehicles	Assign truck	Availability table, capacity/ load compatibility
Employee Management	Maintain valid operational personnel	Add employee	Searchable table, profile panel, deactivate flow
Vehicle Management	Maintain fleet records and state	Add vehicle	Fleet table, status controls, history timeline
Customer Management	Maintain reference data	Add client/reference record	Client table, nested codes/ consignees/locations
Analytics	Review trends and workload	Change reporting period	KPI summaries, charts, drilldown tables
Operational Alerts	Resolve exceptions	Open affected record	Alert inbox, severity, status, timestamps
Users and Settings	Manage access and configuration	Invite user or save setting	User table, role matrix, audit links

7. Detailed Workflow Improvements

7.1 Creating and scheduling trips

Current concept: Trip scheduling and management are handled in different modes of the large `TripList` component, including editor modal state, route stops, map views, and detail tabs.

Recommended change: Use a full-page, structured create/edit workflow rather than a large modal.

Recommended form sections:

1. Trip identity
2. Client and consignee
3. Pickup window and load
4. Route stops
5. Driver, helpers, and vehicle
6. Transfer information, when applicable
7. Review and validation

UX behavior:

- Keep trip code, date, client, and current save state in a sticky header.
- Display completion progress such as “4 of 6 required sections complete.”
- Save draft data where the business process permits.
- Preserve entered values after recoverable errors.
- Validate required fields at the section level and again on submission.
- Show unavailable assets in context, with the reason and conflicting trip.
- Make at least one pickup and one drop-off a visible structural requirement.
- Show source-trip selection only when Transfer is selected.

7.2 Assigning drivers and vehicles

Assignment should occur in a contextual panel containing:

- Asset name or plate
- Current state
- Availability for the selected trip window
- Existing assignments near that time
- Load-type compatibility
- Branch
- Driver licence state where applicable
- Reason unavailable

Recommended behavior:

- Sort available assets first.
- Do not silently remove unavailable options; show them disabled when seeing the reason helps the dispatcher.
- Block selection for inactive assets and confirmed overlaps.
- Revalidate on save to handle concurrent changes.
- Show a clear conflict card:

  Truck ABC-123 is assigned to Trip TA-204 from 09:00–14:00. Choose another truck or adjust the trip window.

Do not provide an override unless stakeholders explicitly approve one.

7.3 Editing existing trips

Use a split-view model:

- Left: trip list or schedule.
- Right: details panel.
- “Edit full trip” opens the full editing page.
- Minor permitted changes can be completed inline or in the panel.

Before saving, summarize consequential changes:

- Driver A will be released.
- Truck ABC-123 will become Available.
- Driver B and Truck XYZ-456 will be assigned.
- An assignment-history entry will be created.

7.4 Updating trip statuses

Use a controlled transition menu rather than an unrestricted status dropdown.

- Display only permitted next states.
- Require confirmation for Cancelled, Transferred, Rescue, and any transition that releases assignments.
- Explain the side effects.
- Require a reason where business or audit requirements apply.
- Do not use a generic “Are you sure?” message.

Example:

> Cancel Trip TA-204?
> This will release Driver Santos and Truck ABC-123 and record the cancellation in the audit and assignment histories.

7.5 Cancellation

The locked behavior releases assignments and records history. The interface should:

1. Show current assignments.
2. Require a cancellation reason.
3. Explain which resources will be released.
4. Prevent duplicate submissions.
5. Confirm the resulting trip and vehicle states.
6. Keep the cancelled trip accessible through filters and history.

7.6 Managing employees and vehicles

Replace destructive “Delete” language with:

- Deactivate employee
- Deactivate vehicle
- Reactivate, where supported
- View history

When deactivation is blocked, provide actionable detail:

> This truck cannot be deactivated because it is assigned to Trip TA-204, currently In Progress.

7.7 Reviewing operational states

Do not rely on a single status filter. Provide saved operational views:

- Today
- Tomorrow
- Unassigned
- In progress
- Requires attention
- Maintenance impact
- Cancelled
- Completed this week

“Requires attention” should be a derived view based on agreed conditions such as:

- Unassigned near departure
- Overdue Scheduled trip
- Assignment conflict
- Inactive assigned resource
- Missing required operational data
- Maintenance vehicle assigned to a future trip

These are UX-derived exceptions, not new authoritative statuses.

7.8 Bulk actions

Appropriate:

- Export selected permitted rows
- Assign a shared filter or saved view
- Deactivate selected inactive-safe reference records, where permitted
- Change non-destructive metadata when business-valid

Avoid bulk trip status changes in the initial MVP unless the transition rules and audit consequences are explicitly defined.

8. Role-Based Access and Interface Behavior

Role behavior

Area	SuperAdmin	Admin	Dispatcher	Encoder	Viewer
Dashboard	Full	Full	Operational	Entry-focused	Read-only
Trip creation	Yes	Yes	Yes	Yes	Hidden
Trip assignment	Yes	Yes	Yes	Hidden	Hidden
Trip updates	Yes	Yes	Yes	Own pending only	Hidden
Trip cancellation	Yes	Yes	Yes	Hidden	Hidden
Vehicle management	Full	Full	Create/update/status	Read-only	Read-only
Vehicle deactivation	Yes	Yes	Hidden	Hidden	Hidden
Employee management	Full	Full	Read-only	Read-only	Read-only
User management	Full	Hidden	Hidden	Hidden	Hidden
Settings	Full	Read-only or scoped	Hidden	Hidden	Hidden
Audit logs	Full	Read	Hidden	Hidden	Hidden

This reflects the official permission matrix and should be enforced by the backend as well as represented in the interface.

Hide versus disable

Hide when:

- The user cannot perform the action under any normal condition.
- The feature contains sensitive administrative information.
- Showing it creates unnecessary cognitive load.

Disable when:

- The action is normally available to the role but blocked by record state.
- The user needs to understand why the action is unavailable.
- The disabled state helps explain a business constraint.

Examples:

- Hide “Invite user” from Admin and operational roles.
- Disable “Deactivate truck” for an Admin when the truck is assigned to an active trip.
- Disable an unavailable driver option and show the conflicting trip.
- Hide edit controls from Viewer users.

Permission feedback

Avoid generic “Access denied” after a user has already invested effort.

- Prevent entry to unavailable workflows.
- Display the current role in the user menu.
- Explain state-based restrictions near the disabled action.
- On server denial, preserve user input and display the permission reason.
- Never infer security from hidden controls; backend authorization remains authoritative.

---

9. Dashboard Recommendations

Dashboard philosophy

The dashboard should answer:

1. What requires attention now?
2. What is scheduled next?
3. Are enough drivers and vehicles available?
4. Where are operations falling behind?
5. What changed recently?

Recommended desktop layout

Top row: operational KPI strip

- Trips today
- In progress
- Unassigned
- At risk
- Available trucks
- Trucks in maintenance

Main left column: exception queue

- Prioritized list of operational issues
- Severity
- Affected trip or resource
- Time remaining

- Recommended next action

Main center/right: schedule horizon

- Today and next 24 hours
- Time-based list or compact timeline
- Assignment completeness
- Status and exception indicators

Lower row

- Fleet availability breakdown
- Recent activity
- Fuel summary
- Completion/cancellation trend

KPI behavior

- Every KPI must drill into its underlying records.
- Include comparison periods only when meaningful.
- Avoid decorative percentages without clear denominators.
- Label timestamps and data freshness.
- Provide empty, partial-data, and error states per widget.

Role-specific dashboards

Dispatcher

- Unassigned
- Departing soon
- In progress
- Availability
- Conflict queue
- Quick create trip

Manager/Admin

- Completion and cancellation
- Fleet utilization
- Maintenance impact
- Fuel cost
- Assignment changes
- Operational trend
- Unresolved exceptions

Encoder

- Own drafts
- Records requiring correction

- Recent trip entries
- Quick create trip

Viewer

- Read-only operational KPIs and recent trips

---

10. Form, Table, Calendar, and Modal Recommendations

Forms

- Use visible labels rather than placeholder-only fields.
- Group fields by task, not database table.
- Mark optional fields explicitly.
- Use lookup values from APIs.
- Use comboboxes for large driver, truck, client, consignee, and location sets.
- Keep validation messages next to fields and summarize blocking errors at the top.
- Provide explicit units for weight, fuel, distance, and time.
- Avoid clearing a form after a failed save.
- Warn before navigating away from unsaved changes.
- Use a sticky action bar on long forms.

Data tables

Trips, vehicles, and employees should default to tables, not card grids.

Recommended table capabilities:

- Persistent search
- Advanced filter drawer
- Sort
- Configurable columns
- Compact and comfortable density
- Saved views
- Pagination
- Row selection where bulk actions are valid
- Sticky header
- Frozen identity column on wide datasets
- Expandable route summary
- Keyboard row navigation
- Side-panel inspection

Carbon’s data-table guidance places global search, filtering, display settings, export, and related utilities in a table toolbar, which is an appropriate model for Cloudy’s operational lists.

Calendars and timelines

Use the schedule for time and resource conflicts—not as a prettier replacement for a table.

Recommended views:

- Day timeline
- Week calendar
- Resource view by truck
- Resource view by driver
- List fallback

Interaction:

- Selecting an event opens details.
- Dragging should not be the only way to reschedule.
- Provide keyboard and menu alternatives for all drag actions.
- Use visual conflict overlays.
- Preserve filters between views.

WCAG requires alternatives to interactions that depend on dragging, and W3C guidance recommends keyboard-operable patterns for complex interactive grids.

Modals

Use modals for:

- Confirming cancellation or deactivation
- Entering a status-change reason
- Small lookup creation
- Simple single-decision tasks

Do not use modals for:

- Full trip creation
- Complex multi-stop editing
- Extensive employee or vehicle records
- Analytics exploration

Side panels

Use a 420–560px contextual panel for:

- Trip quick details
- Assignment selection
- Vehicle history
- Driver availability
- Alert details

- Quick status changes

The panel should preserve the underlying table or schedule context.

---

11. Status, Notification, Validation, and Error-State Design

Status representation

Every status indicator should combine:

- Color
- Text label
- Icon or shape
- Optional supporting description

Suggested semantic mapping:

Meaning	Example	Semantic treatment
Informational future state	Scheduled	Blue label with calendar icon
Active work	In Progress	Strong blue/cyan label with progress icon
Successful completion	Completed, Available	Green label with check icon
Attention required	Maintenance, unassigned, stale	Amber label with warning icon
Operational exception	Rescue, Backload	Orange or violet label with explicit text
Transferred lineage	Transferred	Purple label with transfer icon
Destructive/blocked	Cancelled, conflict, invalid	Red label with stop/error icon
Inactive historical state	Inactive	Neutral gray label with inactive icon

Do not assign green or red merely for visual variety. Their meaning must remain stable.

Alerts

Use four levels:

- **Critical:** active conflict or operation cannot proceed
- **High:** imminent risk requiring action
- **Medium:** incomplete or stale information
- **Informational:** successful or routine system update

Operational alerts should have:

- Clear title
- Affected record
- Trigger time
- Current state
- Recommended action
- Direct link
- Resolution state

Validation timing

- Validate formatting as users leave a field.
- Validate cross-record business rules after relevant fields are selected.
- Revalidate assignments on submission.
- Avoid alarming error states before the user has interacted with a field.
- Do not use toast messages as the only location for form errors.

Error message model

Use:

1. What happened
2. Why it happened
3. How to fix it
4. What was preserved

Example:

> The trip was not saved because Truck ABC-123 was assigned to another trip after you opened this form. Your changes are preserved. Select a different truck and submit again.

System states

Loading

- Skeletons matching the final structure
- Independent widget loading where possible
- Visible progress for saves that may take time

Empty

- Explain whether there is no data or filters produced no matches
- Offer an appropriate next action
- Avoid decorative illustrations on dense operational pages

Success

- Inline confirmation near the changed record
- Short toast for global confirmation
- Updated status visible immediately
- Undo only for genuinely reversible actions

Warning

- Use before consequential but valid actions
- State the effect, not merely “Proceed?”

Error

- Preserve context and entered values
- Include retry where safe
- Show a support/reference ID for unexpected server errors

---

12. Responsive and Accessibility Considerations

Desktop

- Collapsible 240px navigation
- 12-column workspace grid
- Split list/detail views
- Sticky table header and action bars
- Dense operational mode

Tablet

- Navigation rail or overlay drawer
- Single primary workspace plus slide-over details
- 40–48px control heights
- Horizontal table scrolling only where unavoidable
- Schedule defaults to day or agenda view
- Filter controls collapse into a drawer

Material’s adaptive-layout guidance supports changing navigation patterns between compact, medium, and expanded layouts rather than merely shrinking the desktop shell.

Smaller screens

The web MVP should remain usable for urgent inspection but does not need to reproduce every dispatcher workflow.

- Agenda-style trip list
- Record details

- Status viewing
- Limited approved quick actions
- No dense multi-resource schedule
- No complex bulk operations
- No map-first design

Accessibility standards

Target WCAG 2.2 AA.

Key requirements:

- Keyboard access to all actions
- Logical focus order
- Visible focus indicators
- Screen-reader names for icon buttons
- Status announcements after saves
- Accessible dialogs with focus trapping and restoration
- Proper table headers
- Native HTML tables for non-interactive tabular content
- Interactive grid behavior only where spreadsheet-like keyboard navigation is genuinely required
- No color-only meaning
- Text resizing and layout reflow
- Reduced-motion support
- Sufficient contrast in all themes

WCAG 2.2 establishes a minimum pointer-target size of 24×24 CSS pixels, while Cloudy should use larger 40–44px interaction areas for common tablet controls. Focus indicators must remain clearly visible, and interaction-triggered motion should respect reduced-motion preferences.

---

13. Existing Features That Should Remain Unchanged

Existing requirement	Reason to preserve
Hub with disabled Inventory and Billing	Clearly communicates the roadmap without expanding MVP scope
Dashboard trip drill-in	Supports exception investigation
Trip list, calendar, and map concepts	Different views answer different operational questions
Multi-stop trip routes	Core domain requirement
Assignment history	Necessary for operational accountability
Vehicle status history and required reason	Necessary for fleet accountability

Existing requirement	Reason to preserve
Search/filter support	Essential for operational scale
Deactivation instead of hard delete	Preserves historical relationships
Read-only Viewer role	Clear and safe role boundary
Supabase invite/reset flow	Prevents raw-password handling
API-backed lookup values	Prevents frontend and database mismatch
Light and dark theme capability	Useful for office and command-center environments
Backend support for events and fuel	Supports dashboard and later workflow expansion

The current event panel may remain read-only in the first interface iteration unless stakeholders confirm that event entry is required directly from the web UI.

---

14. Existing Features That May Need Modification

Current behavior	Recommended change	Problem solved	Benefit	Trade-off
`TripList` combines many modes and responsibilities	Separate operations, schedule, details, and full editor page concepts	Cognitive and component complexity	Clearer workflows	More routes and shared components
Full trip editor in modal-style state	Use a full-page editor	Insufficient space for multi-stop and assignment context	Fewer errors	More navigation
String-based navigation	Introduce route-level page states	Weak deep linking and refresh behavior	Predictable browser navigation	Architecture work
UI-level role checks	Pair UI permission display with backend authorization	Presentation checks are not security	Correct access behavior	Requires complete API permission mapping
Mock password fields	Remove and use invitations	Conflicts with Supabase Auth	Better security and simpler UX	Requires invitation-status states

Current behavior	Recommended change	Problem solved	Benefit	Trade-off
Hardcoded or mixed status values	Load authoritative lookups	UI/database mismatch	Consistent forms and filters	Requires loading/ fallback behavior
Delete terminology	Replace with deactivate/cancel	Misrepresents domain behavior	Safer actions	Additional states and history
Separate availability discovery	Embed availability in assignment	Context switching	Faster dispatch	More complex assignment component
Generic status dropdown	Use controlled transitions	Invalid state changes	Lower operational risk	Transition rules must be defined
Dashboard dominated by aggregate cards	Add exception queue and schedule horizon	Aggregates do not identify action	Faster intervention	More backend aggregation

15. Missing UX Components or Functional Gaps

Essential gaps

1. **Assignment conflict resolution UI**
2. **Resource availability panel**
3. **Controlled trip-status transitions**
4. **Cancellation side-effect confirmation**
5. **Unsaved-change protection**
6. **Record-changed/concurrency conflict state**
7. **Saved views and persistent filters**
8. **Permission-aware action system**
9. **Audit/history timeline component**
10. **Unified semantic status model**
11. **Form-level validation summary**
12. **Empty, loading, partial, and retry states**

Recommended gaps

- Command palette or global search
- Configurable columns
- Compact/comfortable density toggle
- Alert resolution state

- Recent records
- Keyboard shortcuts
- User-level default view
- Page-level data freshness indicator
- Draft recovery where supported

Optional future enhancements

- Custom dashboard widgets
- Real-time updates
- GPS map monitoring
- Mobile driver workflow
- Automated assignment recommendations
- External notifications
- Report builder
- Predictive delay and maintenance alerts

AI-based analysis should remain supplemental and clearly labeled. It should never obscure authoritative trip, assignment, or status data.

---

16. Prioritized UI/UX Recommendations

Page or workflow	Current issue	Recommendation	Pattern	User benefit	Priority
Trip assignment	Availability is not sufficiently embedded	Contextual driver/truck availability panel with overlap validation	Progressive disclosure	Prevents double-booking	Critical
Trip status	Potentially unrestricted selection	Controlled transition actions with side effects	Guarded action	Prevents invalid states	Critical
Authentication/settings	Mock credential concepts remain in current UI	Remove password management; use invite state	Secure account workflow	Avoids credential risk	Critical
Record deletion	Hard-delete concepts may appear	Use deactivate/cancel terminology and confirmations	Reversible lifecycle	Preserves history	Critical
Trip creation	Complex workflow may be modal-heavy	Full-page sectioned form and sticky summary	Task-focused workspace	Fewer errors	High

Page or workflow	Current issue	Recommendation	Pattern	User benefit	Priority
Trip operations	Filters can become repetitive	Saved views, persistent filters, configurable columns	Personalized workspace	Faster daily use	High
Dashboard	Aggregate cards lack actionability	Exception queue and schedule horizon	Command center	Faster intervention	High
Navigation	Flat view switching	Clear hierarchy and deep-linkable pages	Enterprise app shell	Better orientation	High
Tables	Fixed presentation	Density options, keyboard support, sticky headers	Dense data table	Higher throughput	High
Error handling	Generic alerts	Contextual validation and preserved input	Recoverable error design	Faster correction	High
Permissions	Hidden/ disabled behavior can be inconsistent	Central UI permission policy	Capability-based UI	Lower confusion	High
Availability	Separate driver and truck inspection	Resource timeline and assignment integration	Split view	Fewer context switches	High
Alerts	Exceptions scattered across screens	Operational alert center	Exception inbox	Better follow-through	Medium
Analytics	Dashboard-only interpretation	Drilldown charts and tables	Overview-to-detail	Better management insight	Medium
Global search	Search remains page-specific	Cross-entity search/command palette	Command interface	Faster navigation	Medium
Dashboard widgets	One layout for all roles	Role-specific default layouts	Modular workspace	Better relevance	Medium

Page or workflow	Current issue	Recommendation	Pattern	User benefit	Priority
Advanced customization	User-configurable widgets	Add after workflows stabilize	Bento workspace	Flexibility	Optional
Real-time map	Outside MVP	Defer until GPS exists	Monitoring map	Avoids false expectations	Optional

17. Suggested Next Steps for Wireframing and Prototyping

Phase 1: Workflow definition

Create task flows for:

1. Create an unassigned trip
2. Create and assign a trip
3. Resolve an assignment conflict
4. Reassign a driver and truck
5. Change a trip to In Progress
6. Complete a trip
7. Cancel a trip
8. Change truck status with reason
9. Deactivate an employee
10. Invite a user and assign roles

Phase 2: Low-fidelity wireframes

Prioritize:

- Desktop application shell
- Dispatcher dashboard
- Trip Operations table
- Trip details panel
- Create/Edit Trip page
- Assignment panel
- Day schedule
- Vehicle table and status change
- Employee management
- User/role management

Phase 3: Clickable prototype

Prototype the critical sequence:

Unassigned trip → open details → select truck → conflict shown → select alternative → assign driver → confirm → updated trip and availability state

Phase 4: Usability testing

Test with realistic data volume and at least:

- Two dispatchers
- One operations manager
- One encoder or administrative user

Measure:

- Time to find an unassigned trip
- Time to assign valid resources
- Number of context switches
- Conflict comprehension
- Error recovery
- Status-change confidence
- Ability to locate audit history
- Keyboard-only completion

Phase 5: Design system foundation

Build Figma foundations before high-fidelity page expansion:

- Variables and semantic tokens
- Typography
- Spacing
- Status badges
- Buttons
- Inputs and comboboxes
- Table primitives
- Drawers
- Dialogs
- Toasts
- Alert cards
- Timelines
- Chart styles
- Light/dark/high-contrast themes

---

18. Modern Web Application Design Ideas for 2026

Patterns to adopt

Exception-driven operations interfaces

Surface abnormal or actionable conditions rather than giving all records equal weight.

Cloudy application:

- Unassigned trips near departure
- Overlapping assignments
- Trucks in maintenance that affect future trips
- Trips that have not progressed as expected
- Inactive resources connected to future work

Split-view list and details

Maintain queue context while inspecting a record.

Cloudy application:

- Trip table plus quick details
- Vehicle table plus status history
- Alert inbox plus affected record
- Availability list plus assignment context

Progressive disclosure

Show essential data first and reveal complex history, fuel, events, and secondary fields when required.

Saved views and configurable tables

Frequent users should be able to retain combinations of filters, sorting, density, and visible columns.

Adaptive navigation

Use a full sidebar on expanded layouts, navigation rail or drawer on medium layouts, and a reduced mobile information architecture.

Design tokens

Tokens should define color, spacing, typography, border, elevation, and motion rather than isolated page values. Fluent’s token model explicitly supports a common design-development language across platforms, while Carbon provides enterprise-focused components and documentation suitable for data-heavy products.

Purposeful skeleton loading

Skeletons should reflect the final table, card, or panel shape and allow independent areas to load without blocking the entire screen.

Functional microinteractions

Use motion to communicate:

- Panel opening context
- Row update
- Successful assignment
- Filter application
- Status transition

Keep motion short, consistent, and removable through reduced-motion preferences. Fluent describes motion as a mechanism for communicating relationships and transitions rather than decoration.

Patterns to adapt

Design idea	Cloudy use	Risk	Recommendation
Bento dashboards	Organize role-specific widgets	Fragmented scan path	Use a restrained grid with fixed priority zones
Inline editing	Low-risk fields and assignment actions	Accidental changes	Limit to reversible, permission-safe fields
Command palette	Navigation and record search	Discoverability	Add visible global-search entry and shortcut
Optimistic updates	Low-risk metadata	Misleading state when backend rejects	Use only where rollback is clear
Map view	Route context and later GPS	Implies real-time tracking	Label static route context clearly
Custom widgets	Manager preferences	Inconsistent shared operations view	Defer until core metrics stabilize

Use sparingly

- Glass effects: login or decorative shell surfaces only; avoid under data.
- Gradients: brand accent areas, never status meaning or table backgrounds.
- Large rounded cards: moderate radius only.
- Oversized typography: login and empty states, not operational pages.
- Animated transitions: 120–200ms for state continuity.
- Floating controls: only a clearly scoped tablet action, not primary desktop navigation.

Avoid

- Floating desktop navigation
- Highly minimal interfaces that hide labels
- Large areas of decorative whitespace
- Low-density card dashboards
- Status represented only by colored dots
- Continuous animated maps or charts
- Excessive blur and transparency
- Hidden filters with no active-filter summary
- Icon-only primary actions
- Drag-only scheduling
- Custom interaction patterns that ignore keyboard conventions

Future driver application considerations

The web design should prepare shared domain patterns:

- Same status names and semantic colors
- Same trip identity hierarchy
- Same stop timeline
- Same assignment and event terminology
- Shared notification severity
- Shared design tokens where platform-appropriate

Do not force the desktop information architecture into the future driver app. Driver workflows should instead focus on assigned trip, next stop, actions, proof/status events, and support.

---

19. Comparison of Proposed Visual Design Directions

Criteria	A. Modern Operations Console	B. Fleet Command Center	C. Modular Management Workspace
Overall concept	Task and queue oriented	Monitoring and exception oriented	Configurable role workspace
Visual style	Structured, restrained, dense	High-contrast status and visualization	Card/panel grid
Information density	High	Medium-high	Variable
Navigation	Persistent sidebar	Sidebar plus monitoring modes	Sidebar plus dashboard customization
Dashboard	Operational tables and compact KPIs	Exceptions, map, KPIs, live activity	User-selected widgets

Criteria	A. Modern Operations Console	B. Fleet Command Center	C. Modular Management Workspace
Tables	Primary interface	Supporting detail	Embedded in modules
Forms	Structured full-page forms	Secondary to monitoring	Modular sections
Scheduling	Excellent	Good	Good
Monitoring	Good	Excellent	Good
Manager suitability	High	Very high	Very high
Dispatcher suitability	Very high	High	Medium-high
Learning curve	Low-medium	Medium	Medium-high
Accessibility	Strong if conventional components are used	Strong if visualizations have textual alternatives	Variable due to customization
Tablet behavior	Good with split-view collapse	Good with agenda and alert modes	Good if widgets reflow
Implementation complexity	Moderate	Moderate-high	High
Maintainability	High	Medium-high	Medium
Main advantage	Fast, predictable daily work	Excellent situational awareness	Role and user flexibility
Main risk	Can feel utilitarian	Can become decorative or map-heavy	Can lose consistent priority
Best pages	Trips, forms, vehicles, employees	Dashboard, alerts, schedule monitoring	Manager dashboard, analytics
MVP suitability	Excellent	Good as a dashboard layer	Limited initial use

20. Recommended Final Design Direction

Hybrid: Operational Command Workspace

Use **Modern Operations Console** as the foundation, add **Fleet Command Center** patterns to the dashboard and schedule, and postpone substantial user-configurable workspace behavior.

Visual identity

- Professional, restrained, and operational
- Neutral layered surfaces
- Strong typography and alignment
- Moderate density
- Semantic accent colors
- Minimal decorative effects
- Clear boundaries between navigation, data, and actions

Why this is appropriate

1. Dispatch is the highest-frequency workflow.
2. The MVP contains significant tabular and form-based operations.
3. Real-time GPS is not present, so a map-dominant command center would imply functionality that does not exist.
4. Managers still require strong monitoring and exception visibility.
5. A fully customizable dashboard would add implementation and governance complexity before core workflows have stabilized.
6. The current React application already contains table, calendar, map, chart, and detail-panel concepts that can be reorganized into this hybrid.

Recommended distribution

- **70% Operations Console:** trips, assignments, employees, vehicles, settings
- **25% Command Center:** dashboard, alerts, schedule monitoring
- **5% Modular Workspace:** role-specific default widgets and saved views

---

21. Page-Level Application of the Recommended Design

Login

Objective: Securely enter Cloudy.

Layout: Centered authentication panel with product identity and restrained supporting visual.

Components: Email, password, show/hide control, sign-in button, reset-password link, error banner.

States: Signing in, invalid credentials, rate-limited, session expired, service unavailable.

Tablet: Same centered layout.

Dashboard

Objective: Identify operational risk and capacity.

Layout: KPI strip, exception queue, schedule horizon, fleet availability, recent activity.

Primary action: Open the highest-priority exception.

Secondary actions: Create trip, open Today view, inspect availability.

Desktop: Two- or three-column composition.

Tablet: Single prioritized feed; secondary widgets below.

Trip Schedule

Objective: Understand trips and resources over time.

Layout: Header filters, day/week/resource switcher, timeline or calendar, details panel.

Primary action: Create trip.

Secondary actions: Open trip, assign resources, change view.

States: No trips, no matches, partial load, conflict overlay, API error.

Tablet: Default to day agenda; details open as full-height drawer.

Create Trip

Objective: Create a valid, auditable trip.

Layout: Main sectioned form plus sticky right summary/validation panel.

Primary action: Create trip or save changes.

Secondary actions: Save draft, cancel, preview route.

Information hierarchy: Identity → date/load → route → assignment → review.

Tablet: Single-column form with collapsible summary and sticky footer.

Trip List / Operations

Objective: Manage the operational queue.

Layout: Saved-view tabs, search/filter toolbar, dense table, contextual panel.

Primary action: Create trip.

Secondary actions: Assign, update status, export, configure columns.

States: Empty dataset, no filter results, loading rows, stale data, permission denial.

Tablet: Reduced columns and full-height details drawer.

Trip Details

Objective: Understand a trip's current state and history.

Layout: Header with identity/status, summary grid, route, assignments, timeline.

Tabs: Overview, Stops, Assignments, Events, Fuel, Activity.

Primary action: Current valid next action.

Secondary actions: Edit, reassign, cancel, transfer, view history.

Desktop: Full page with sticky action header.

Tablet: Stacked sections and overflow action menu.

Driver Availability

Objective: Find a valid driver for a time window.

Layout: Date/window controls, availability summary, table or resource timeline.

Columns: Driver, role, branch, licence, current state, next assignment, availability reason.

Primary action: Assign selected driver when entered from a trip.

Tablet: Availability list with expandable assignment details.

Vehicle Availability

Objective: Find a valid truck.

Layout: Similar to driver availability, with load and capacity filters.

Columns: Plate, status, load compatibility, branch, current/next trip, availability.

Primary action: Assign truck.

Warnings: Maintenance, inactive, overlap, incompatible load, capacity concern where defined.

Employee Management

Objective: Maintain operational personnel.

Layout: Search/filter table and employee profile panel.

Primary action: Add employee.

Secondary actions: Edit, deactivate, view availability/history.

States: Driver licence missing, active assignment blocks deactivation, inactive record.

Vehicle Management

Objective: Maintain vehicle records and availability state.

Layout: Fleet table, vehicle detail panel, status-history timeline.

Primary action: Add vehicle.

Secondary actions: Edit, change status, deactivate.

Confirmation: Status changes require reason; deactivation explains blocking assignments.

Customer Management

Objective: Maintain clients and related reference data.

Layout: Client list plus nested tabs for internal codes, consignees, and locations.

Primary action: Add client.

Secondary actions: Add code, consignee, or location.

Role: Admin/SuperAdmin management; operational roles receive read access where required.

Analytics and Reports

Objective: Review operational patterns.

Layout: Date and client filters, KPI summary, limited charts, drilldown table.

Charts: Completion/cancellation trend, fleet state, fuel trend, workload distribution.

Primary action: Change reporting scope.

Secondary action: Export only when approved.

MVP limitation: No custom report builder.

Operational Alerts

Objective: Triage and resolve operational exceptions.

Layout: Filterable alert inbox and details panel.

Primary action: Open affected trip/resource.

Secondary actions: Mark reviewed or resolved where meaningful.

States: No alerts, stale alerts, affected record changed, alert no longer applicable.

User and System Settings

Objective: Manage authorized users and safe configuration.

Layout: Tabs for Users, Roles/Permissions, App Settings, Audit Log.

Primary actions: Invite user; assign role; save setting.

Prohibited UI: Raw password fields.

Permission presentation: Matrix by module and action, with inherited role permissions clearly distinguished from user-specific adjustments if those are supported.

---

22. Suggested Design System and Component Standards

Foundations

Color tokens

Base

- `surface-canvas`
- `surface-primary`
- `surface-secondary`
- `surface-raised`
- `surface-overlay`
- `border-subtle`
- `border-default`
- `text-primary`
- `text-secondary`
- `text-muted`
- `text-inverse`

Brand

- `brand-primary`
- `brand-primary-hover`
- `brand-primary-active`
- `brand-subtle`

Semantic

- `status-info`
- `status-active`
- `status-success`
- `status-warning`
- `status-danger`
- `status-neutral`
- Each requires text, background, border, and icon variants.

Do not use raw palette values directly in page designs.

Typography

Use Inter or another highly legible UI sans-serif.

Suggested scale:

Token	Size/line height	Use
Display	32/40	Login or major empty state only
Heading 1	24/32	Page title
Heading 2	20/28	Major section
Heading 3	16/24 semibold	Card/panel title
Body	14/20	General UI text
Body compact	13/18	Dense tables
Label	12/16 semibold	Form and metadata labels
Caption	12/16	Supporting metadata

Clear typographic hierarchy is essential for helping users locate structure and priority in complex applications.

Spacing scale

Use a 4px base:

- 4, 8, 12, 16, 20, 24, 32, 40, 48, 64

Operational pages should use:

- 8–12px internal table spacing
- 16px card/panel spacing
- 24px section spacing
- 32px page-region spacing

Radius

- 4px: compact controls and status labels
- 6px: inputs and buttons
- 8px: panels and tables
- 12px: large dashboard cards or dialogs
- Avoid pill shapes except badges, tags, and segmented controls

Elevation

Use borders before shadows.

- Level 0: canvas
- Level 1: standard panel with border
- Level 2: sticky header or raised panel
- Level 3: drawer or dropdown
- Level 4: modal

Dark mode should avoid excessive glowing shadows.

Grid and layout

- 12-column expanded grid
- 8-column medium grid
- 4-column compact grid
- Maximum content width only on forms and settings
- Operational tables may use the full available workspace width

Icons

- Use one consistent line-icon family.
- Default 16 or 20px.
- Pair ambiguous icons with labels.
- Provide tooltips for secondary icon buttons.
- Do not use truck, user, or status icons as the sole differentiator.

Button hierarchy

1. Primary: one dominant page or panel action
2. Secondary: common supporting action
3. Tertiary/ghost: low-emphasis action
4. Danger: destructive or high-consequence action
5. Icon button: compact supporting action with accessible name

Form controls

- Standard height: 40px desktop, 44px tablet
- Compact variant: 32–36px only for dense desktop toolbars
- Labels always visible
- Help and error text occupy reserved space where practical
- Combobox search for large lookups
- Date/time controls expose timezone context
- Required state not communicated by color alone

Data tables

Reusable variants:

- Standard
- Compact operations
- Selectable/batch
- Expandable
- Read-only
- Interactive grid only when advanced keyboard navigation is required

W3C encourages native HTML tables when possible and reserves grid semantics for truly interactive tabular widgets.

Cards

Use cards for:

- KPI summaries
- Exceptions
- Availability summaries
- Recent activity
- Empty states

Do not replace straightforward tabular datasets with repeated cards.

Drawers and side panels

- 420px standard
- 560px complex detail
- Full-width overlay below medium breakpoints
- Header, scrollable body, sticky footer actions
- Return focus to triggering control when closed

Toasts and alerts

- Toasts confirm transient success.
- Inline alerts communicate errors requiring action.
- Banners communicate page- or system-level conditions.
- Critical operational alerts remain visible until addressed or no longer valid.

Tabs

Use tabs for closely related record-level sections such as Overview, Stops, Assignments, Events, Fuel, and Activity. Tabs should not be used as a substitute for primary application navigation. Carbon similarly positions tabs as a way to group related content within pages, cards, modals, and side panels.

Charts

- Always pair charts with values or a drilldown table.
- Use the same semantic colors as the status system.
- Limit simultaneous series.
- Avoid 3D charts.
- Avoid animated chart entrances on repeated dashboard visits.
- Provide textual descriptions and accessible labels.

Loading indicators

- Skeleton for initial content
- Inline spinner for single actions
- Progress indicator only for multi-step or extended operations
- Prevent duplicate submissions while preserving context

Empty states

Every empty state should identify one of:

- No records exist
- No records match filters
- Data is not available
- User lacks access
- A feature is coming later

Focus

- Minimum visible 2px high-contrast outline
- Never remove focus without an accessible replacement
- Use `:focus-visible`
- Restore focus after drawers and dialogs close

Motion

- 120–160ms for hover and small state changes
- 160–220ms for panels and dialogs
- No large parallax or spring effects
- No motion required to understand state
- Respect reduced-motion settings
- Use opacity changes carefully in dense tables so text does not become unreadable

Theme support

Light mode

- Default for conventional office environments
- Neutral canvas with strong table boundaries

Dark mode

- Useful for prolonged monitoring environments
- Avoid pure black
- Maintain status contrast
- Increase border visibility rather than relying on shadows

High-contrast mode

- Independent semantic token set
- Strong outlines
- Reduced decorative surfaces
- Status labels with icons and explicit text
- Tested with browser and operating-system forced-color behavior

---

Final Recommendation

Cloudy should be designed as an **Operational Command Workspace**:

- The trip queue is the center of dispatcher work.
- The dashboard is the center of managerial awareness.
- Availability is part of assignment, not a disconnected lookup.
- Exceptions outrank decorative metrics.
- Tables are the principal operational component.
- Side panels preserve context.
- Full pages support complex editing.
- Status transitions and destructive actions explain their consequences.
- Role restrictions remain clear but unobtrusive.
- The design system uses reusable semantic tokens and conventional, accessible interaction patterns.

The first prototype should focus on one end-to-end operational scenario: finding an unassigned trip, checking availability, encountering a conflict, selecting valid resources, confirming the assignment, and verifying the resulting trip, driver, truck, and history states.